

Computer-Use Automation System

Implementation and Design Plan - interface.ai Engineering Take-Home

Assignment	Take-Home Project: Computer-Use Automation System (interface.ai)
Deliverable	Detailed design + implementation plan, justified against the published evaluation criteria
Stack	TypeScript + Node.js 22, Playwright, LLM provider adapter (Anthropic + OpenAI), Zod
Target surface	Locally built mock credit-union back-office app (deliberately legacy-hostile)
Date	September 2026
Status	Design baseline v1.0 - ready for implementation

The through-line this design is built around

"The model discovers. The artifact becomes a reusable capability. Deterministic replay is how the AI agent invokes it in production."

Every component below exists to serve one of those three phases - or to keep them safe (policy), observable (evidence), or recoverable (escalation).

How to read this document. Each major decision is presented as: the decision, why it wins, the alternatives it beat and why they lose, and the evaluation criterion it serves. Section 2 maps the whole design onto the grading rubric; Sections 5-8 are the load-bearing pieces (artifact schema, deterministic replay, escalation, safety); Section 14 is the build plan.

Contents

1. Executive summary	4
1.1 Positioning against the brief	4
1.2 Decisions at a glance	4
1.3 What will be demonstrated	4
2. Requirements digest and evaluation mapping	6
2.1 Core requirements coverage (brief Section 3)	6
3. System architecture	7
3.1 Modules and dependency rules	7
3.2 The three data flows	8
3.3 Why a modular monolith (and not services)	8
4. Technology decisions and rejected alternatives	9
4.1 The discovery loop, concretely	9
4.2 Where typed inputs, outputs, and outcomes come from	10
5. Artifact schema - the capability contract	11
5.1 Shape	11
5.2 A step, annotated	11
5.3 Target descriptors: ranked, semantic-first locator strategies	11
5.4 Conditions: a small declarative AST	12
5.5 Parameterization and secrets	12
5.6 Versioning and review lifecycle	12
5.7 Alternatives rejected	13
6. Deterministic replay and the error taxonomy	14
6.1 Execution algorithm	14
6.2 Wait and checkpoint strategy	14
6.3 The three-class taxonomy, applied to the brief's own conditions	14
6.4 Result contract	15
6.5 What determinism means here	15
7. Escalation and human-in-the-loop handoff	16
7.1 Detecting "stuck"	16
7.2 Control-ownership state machine	16
7.3 The intervention request	16
7.4 Taking control of the live session	17
7.5 Handback and resume semantics	17
8. Safety and data handling	18

8.1 One chokepoint, physically unavoidable	18
8.2 The allowlist model (deny by default)	18
8.3 Risky and irreversible actions	18
8.4 Redaction pipeline (regulated financial data)	18
8.5 Prompt-injection stance and honest limits	18
8.6 policy.yaml (excerpt; full file in Appendix C)	19
9. Evidence and observability	20
10. Heterogeneity and multi-tenant reuse (design)	20
10.1 The surface seam	20
10.2 Multi-tenant reuse: base artifact + tenant binding overlay	21
10.3 Detecting and managing drift	21
11. The mock target application ("CU BackOffice")	22
11.1 Flows	22
11.2 Fault-injection matrix (query flag or config)	22
12. Testing strategy - where it counts	23
13. Repository layout and deliverables mapping	23
13.1 Demo path (verbatim in README)	23
14. Implementation roadmap, cut lines, and stretch goals	24
14.1 Phases (est. 28-36 focused hours total)	24
14.2 Cut lines - deliberate, with the seam designed	24
14.3 Stretch goals - at most two, in this order	24
14.4 What gets built next with more time	25
15. Risk register	25
Appendix A - Complete example artifact (member-savings-lookup)	26
Appendix B - Result contract examples	29
Appendix C - policy.yaml	29

1. Executive summary

This plan describes **Scribe**, a small but complete computer-use automation system for the interface.ai take-home. Scribe does four things end to end: (1) an LLM-driven agent accomplishes a natural-language goal against a live UI; (2) a **Recorder** distills that successful run into a typed, versioned, human-reviewable **capability artifact**; (3) a **replay engine** executes that artifact deterministically - no LLM in the decision loop - with an explicit error taxonomy separating business outcomes, recoverable conditions, and hard failures; and (4) when it cannot proceed safely, it **escalates to a human** who takes control of the same live browser session and hands control back.

The system is a **modular monolith in TypeScript**: one package, ten modules with enforced dependency boundaries, driven by a CLI plus a minimal local operator console. It automates a locally built, deliberately legacy-style **mock credit-union back-office** so that the exact runtime failures the brief cares about (record not found, validation errors, permission denials, session expiry, interstitial dialogs, slow loads) can be reproduced on demand and demonstrated in committed evidence.

1.1 Positioning against the brief

The brief is explicit about where the difficulty lives: the artifact schema and replay contract are "a focal point of the evaluation", conflating business outcomes with failures is "the most common design mistake", and escalation must be "real, not just a TODO". This design therefore spends its complexity budget on three load-bearing pieces - the artifact schema (Section 5), the replay engine and its deviation classifier (Section 6), and the control-transfer model (Section 7) - and deliberately keeps everything else thin: no services, no queues, no database, filesystem JSON stores, a single-page operator console.

1.2 Decisions at a glance

Axis	Decision	One-line rationale
Language / runtime	TypeScript on Node.js 22	The rubric's focal point is a typed schema; Zod gives one source of truth for static types and runtime validation.
Automation substrate	Playwright (Chromium, headed)	Actionability auto-waits, first-class iframes, accessibility-tree snapshots, CDP access for live handoff, built-in tracing.
Perception	Accessibility-tree-first hybrid (+ screenshots, bbox fallback)	Works when the DOM is hostile; same abstraction exists on desktop; the brief says to bias toward no-clean-DOM approaches.
LLM	Provider adapter: Anthropic (default) + OpenAI	Tool-calling loop with strict JSON action schemas; provider is a seam, not a dependency.
Target app	Local mock CU back-office with fault injection	Full control over the error states the rubric grades; offline for reviewers; banking-realistic flows.
Architecture	Modular monolith + CLI + local operator console	The brief explicitly does not reward scaling infrastructure; boundaries live in module contracts.
Artifact store	Versioned JSON files in git	Reviewable in PRs (the review workflow IS the approval workflow); zero setup for reviewers.

1.3 What will be demonstrated

- **Discovery**: a real LLM-driven run completing "Look up member 12345 and read their current savings balance" against the live mock app, with evidence.
- **Replay (happy path)**: the saved artifact re-run with params, no LLM, checkpoint-verified, returning typed outputs.
- **Replay (business outcome)**: memberId=99999 returns MEMBER_NOT_FOUND as a legitimate, structured result - not an error.
- **Replay (recovery)**: an injected interstitial dialog is detected and dismissed by a declared recovery; a slow load is retried within bounds.

- **Escalation:** an injected session expiry exhausts recovery, raises an intervention, a human takes over the live browser, logs in manually, hands back, and the run resumes and completes - all captured in evidence.
- **Safety:** allowlist enforcement blocking an off-policy navigation, a risky (mutating) step gated on approval, and redaction of sensitive values in artifacts, logs, and screenshots.

2. Requirements digest and evaluation mapping

The graders state their weighting order explicitly. The table below maps each criterion to the design element that answers it and the section that argues it. This is the contract this document is written against.

#	Evaluation criterion (in stated weight order)	Design answer	Where
1	System design - boundaries, data models, trade-offs, simplicity; artifact schema and replay contract are central	Modular monolith with three seams (surface, provider, policy); artifact split into contract / mechanics / policy; typed result contract	Sec. 3, 5
2	Correctness of the core loop - agent completes a real goal; artifact replays deterministically and verifies success	Tool-calling discovery loop; Recorder distillation; checkpoint-verified replay with no LLM import path	Sec. 4, 6
3	Robustness and error handling - runtime errors vs business outcomes vs hard failures; locator, wait, checkpoint strategy	Declared outcomes + declared recoveries + fail-closed hard failures, classified in strict precedence; ranked locators; bounded condition waits	Sec. 6
4	Human-in-the-loop escalation - detect stuck, route with context, transfer the live session, resume	Control-ownership state machine (AGENT/PAUSED/HUMAN); intervention store; same headed browser operated by the human; recorded human actions; disposition-based resume	Sec. 7
5	Generalization - heterogeneous surfaces; artifact reuse across tenants without rebuilds	SurfaceDriver seam + surface-agnostic TargetDescriptor and condition AST; base artifact + per-tenant binding overlay; drift telemetry	Sec. 10
6	Safety and data handling - allowlist, risky actions, redaction	PolicyEngine chokepoint inside the driver; risk classes with approval gates; parameterization strips PII from artifacts; masked logs and screenshots	Sec. 8
7	Code quality - readable, typed, tested where it counts	Strict TS + Zod; dependency-boundary lint; tests focused on schema, policy, classifier, locators	Sec. 12
8	Communication - reasoning, trade-offs, cut lines	REPORT.md with the seven required headings; explicit cut lines	Sec. 13, 14

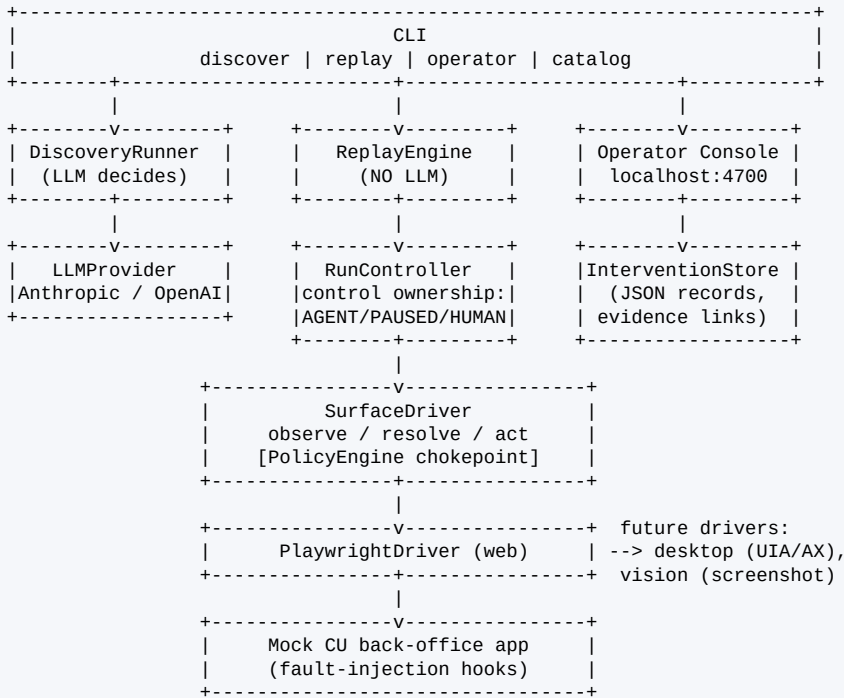
2.1 Core requirements coverage (brief Section 3)

Requirement	Coverage in this design
3.1 Goal-driven agent loop	Discovery runner: observe (a11y snapshot) - decide (LLM tool call) - act (driver), with stop conditions: goal met, max steps, timeout, loop detection, model-declared stuck.
3.2 Structured artifact	CapabilityArtifact v1: typed inputs/outputs, ranked target strategies, per-step checkpoints, declared outcomes and recoveries, policy block, provenance. Section 5.
3.3 Deterministic replay	Replay engine with per-step algorithm, deviation classifier, four-state result contract, bounded waits, fail-closed ambiguity handling. Section 6.
3.4 Safety guardrails	Deny-by-default allowlist, risk classes with conservative gates, no secrets or raw PII persisted. Section 8.
3.5 Evidence / observability	Per-run JSONL event log with actor attribution + screenshots at steps, checkpoints, and failures. Section 9.
3.6 Escalation and handoff	Intervention requests with context; live session control transfer; human-action recording; resume semantics. Section 7.
3.7 Heterogeneity and scale (design)	Surface abstraction and multi-tenant overlay model; drift detection. Section 10.

3. System architecture

Serves evaluation criteria: System design (criterion 1) - clear boundaries, appropriate simplicity.

Scribe is a single TypeScript package organized as a **modular monolith**: ten modules with explicit, lint-enforced dependency rules, wired together by a CLI. There are no services, queues, or databases - by design, and the brief says plainly that building scaling infrastructure is not rewarded. The boundaries that matter for the real environment exist as **interfaces** (seams), each aligned to an axis of change: the surface you automate, the model provider you call, and the policy you enforce.



cross-cutting: core/ (Zod schemas + types) is imported by every module;
evidence/ (redacting logger, screenshots, run directories)

3.1 Modules and dependency rules

Module	Responsibility	May depend on
core/	Zod schemas + inferred types: artifact, result contract, condition AST, policy config, intervention records. Pure data.	nothing
surface/	SurfaceDriver interface + PlaywrightDriver. Observation (a11y snapshots), target resolution, actions. Hosts the policy chokepoint.	core, policy, evidence
llm/	LLMProvider interface + Anthropic and OpenAI implementations; strict JSON tool schemas for agent actions.	core
agent/	Discovery loop (observe-decide-act), stop conditions, Recorder (distills run into an artifact, parameterizes literals).	core, surface, llm, policy, evidence, escalation
replay/	Deterministic executor: waits, resolution, checkpoints, DeviationClassifier, recoveries, result contract. No import path to llm/ - determinism by construction, enforced by lint.	core, surface, policy, evidence, escalation
policy/	PolicyEngine: allowlist, action-kind rules, risk classification, approval gates.	core
escalation/	RunController (control-ownership state machine), intervention store, operator console server, human-action recorder.	core, evidence

Module	Responsibility	May depend on
evidence/	Redacting structured logger (JSONL), screenshot capture, run directory management.	core
target-app/	Mock CU back-office (Express + EJS) with fault-injection hooks. Runs standalone; no imports from the system.	nothing
cli/	Command wiring: discover, replay, operator, catalog.	all

3.2 The three data flows

- **Discovery:** goal + example params in - the runner loops (snapshot - LLM decision - policy check - act - record) until the model marks success or a stop condition fires; the Recorder then distills the action trace into a CapabilityArtifact, replacing literal values with parameter references, and writes it to artifacts/ as reviewable JSON.
- **Replay:** artifact + params in - preflight (validate params with Zod, load policy, verify app fingerprint) - per-step execute-and-verify - on deviation, classify (outcome / recovery / hard failure) - typed result out. No model calls anywhere on this path; it runs with no API key configured.
- **Escalation:** any phase can raise an intervention - the RunController flips control to PAUSED and parks the automation on a control mutex - the operator claims it in the console, drives the same headed browser, and resumes with a disposition - the engine re-verifies state and continues, with every human action recorded in the same evidence stream.

3.3 Why a modular monolith (and not services)

Decision: one process per run, seams as interfaces, filesystem stores

Why. A discovery or replay run is a single, short-lived, stateful session bound to one live browser - there is no concurrency problem for a take-home to solve, and the rubric explicitly withholds credit for queues, clusters, and multi-tenant plumbing. Every boundary that would matter at scale is expressed as a TypeScript interface with a dependency rule, which is what actually gets evaluated: can the design grow without rewrites?

Rejected: microservices / job queue. Adds deployment and IPC complexity that demonstrates nothing the rubric measures; hides the seams inside network plumbing instead of readable contracts.

Rejected: SQLite/Postgres for artifacts and interventions. A database adds reviewer setup friction and makes artifacts less reviewable. JSON files in git mean an artifact diff is a PR diff - which doubles as the review/approval workflow the artifact lifecycle needs (Section 5.6).

Production path (documented, not built). Each module maps 1:1 to a service later: driver pool per tenant, artifact registry, intervention queue + operator UI. The interfaces are the migration plan.

4. Technology decisions and rejected alternatives

Serves evaluation criteria: System design, correctness of the core loop, code quality.

Decision	Why it wins	Rejected alternatives and why
TypeScript + Node.js 22	The artifact schema is the rubric's focal point: Zod provides one definition yielding both static types and runtime validation of every artifact, param set, and result. Discriminated unions model the result contract natively. One language across agent, replay, console, and target app.	Python + Pydantic: entirely viable, but Playwright is TS-first, and splitting languages (Python system, JS console/app) weakens cohesion. Go/Java: slower iteration, weaker browser-automation ecosystem.
Playwright	Actionability auto-waits (visible/enabled/stable) remove the sleep-based flake that kills deterministic replay; first-class frame support (legacy framesets); accessibility snapshots for perception; CDP access underpins the live-session handoff; built-in tracing and screenshots feed evidence.	Selenium: manual wait discipline, flakier under dynamic content. Puppeteer: Chromium-only, thinner a11y and locator APIs. Computer-use / browser-agent SDKs: they hide exactly the loop, locator, and policy design this assignment evaluates, and their actions bypass our policy chokepoint.
Accessibility-tree-first perception	The brief: bias toward approaches that survive a hostile DOM. The agent observes a numbered snapshot of role/name/value/state/bbox nodes - not raw HTML - so decisions attach to semantics a human would use. The same abstraction exists on desktop (UIA/MSAA, macOS AX), which is the generalization story. Screenshots supplement when structure is ambiguous.	Raw DOM/CSS-first: brittle on server-rendered legacy markup, dead end for desktop. Pure screenshot + coordinates: universal but brittle to layout shifts and poor for review; retained only as the recorded last-resort strategy (bbox + element description).
LLM provider adapter (Anthropic default, OpenAI alternate)	Decisions come back as strict, schema-validated tool calls (click/fill/extract/...), each carrying machine-useable arguments plus human-readable intent and reasoning - which is what makes the recorded artifact reviewable. Temperature 0. The provider is a seam: one interface, two implementations, swappable by flag.	Hardcoding one vendor: cheaper but forfeits the seam. Free-text prompting + parsing: fragile, unvalidatable, and worse for recording structured steps.
Local mock CU back-office as target	The rubric grades how replay handles runtime states: not-found, validation errors, permission denials, session expiry, interstitials, slow loads. A local app makes each reproducible on demand via injection flags - so the committed evidence can show every class. Banking-shaped flows (member search, balances, sub-account open) mirror the brief's own examples. Offline and ToS-clean for reviewers; enables a tenant-B variant later.	Public demo sites (saucedemo, demoqa): zero build cost but cannot produce session expiry or permission denials on demand, which guts the robustness demonstration; ToS and rate-limit caveats; no domain realism.

4.1 The discovery loop, concretely

Each turn, the runner sends the model: the goal, declared input parameters (names + example values), a compact numbered accessibility snapshot (ref, role, name, value, state per node, plus URL and title), the last action's result, and optionally a screenshot. The model must respond with exactly one tool call from a closed set:

```
click(ref, intent, reasoning)      fill(ref, value|{{param}}, intent, reasoning)
select(ref, option, ...)           press(key, ...)
navigate(url, ...)                 extract(name, ref, type, ...)
declare_outcome(code, detectorHint) mark_success(summary)
escalate(reason)                  wait_for(condition, timeoutMs)
```

Stop conditions: mark_success, max steps (default 25), wall-clock timeout, a repeated (action, state-hash) pair observed three times (loop detection), or escalate. Every executed action plus the observation that followed it is appended to the run trace that the Recorder distills. The model never executes anything itself: every action passes through the same SurfaceDriver - and therefore the same PolicyEngine - that replay uses.

4.2 Where typed inputs, outputs, and outcomes come from

- **Inputs:** declared at the CLI at discovery time (e.g. memberId=12345). The Recorder replaces exact literal occurrences in fills and URLs with {{inputs.memberId}} - parameterization doubles as PII stripping.
- **Outputs:** the agent's extract(name, ref, type) calls declare them; the Recorder captures the target descriptor and declared type into the outputs block.
- **Outcomes:** a happy-path run never sees "member not found", so outcomes come from two honest sources: optional **probe runs** (the CLI accepts exemplar bad inputs, e.g. memberId=99999; the agent runs until the outcome state appears and records a detector for it) and **human review** of the artifact JSON, which is designed to be hand-editable. No hallucinated detectors: every detector is grounded in an observed state or a reviewer edit.

5. Artifact schema - the capability contract

Serves evaluation criteria: System design (the stated focal point), robustness, generalization, safety.

The artifact is the product of discovery and the input to replay - the brief calls it a capability an AI agent can call, and says to design the schema deliberately. The single organizing idea here: **separate the contract from the mechanics from the policy**. A calling agent needs only the contract (what to send, what comes back, which business outcomes exist). The replay engine consumes the mechanics (steps, targets, waits, checkpoints). The policy engine reads the policy block. A human reviewer reads all three - which is why every step keeps the model's stated intent as prose.

5.1 Shape

```
CapabilityArtifact (v1)                                // stored as artifacts/<id>@<version>.json
|- schemaVersion: "1.0"                               // format version -> migration path
|- capability: { id, name, description, version (semver), tags[] }
|- target: { appId, surface: "web", endpoint (templated URL),
|           appFingerprint: { titlePattern, markers[] } } // fail fast on drift
|- inputs[]: { name, type, required, pattern?, sensitive, description, example? }
|- outputs[]: { name, type, description, sensitive, sourceStep }
|- steps[]: { id, intent, action, target?, value?, waitBefore?, checkpoint,
|            risk: "safe"|"risky", onDeviation?[] }
|- outcomes[]: { code, description, terminal, detector: Condition, extract?[] }
|- recoveries[]: { id, detector: Condition, action, maxAttempts, appliesTo }
|- successCheckpoint: Condition
|- policy: { requiredOrigins[], allowedActionKinds[], riskLevel,
|           unattendedReplay: boolean }
|- provenance: { provider, model, runId, recordedAt, reviewStatus: "draft"|"approved" }
```

Appendix A contains a complete, internally consistent artifact for the demo capability member-savings-lookup. Everything is Zod-defined in core/; the static types are inferred from the same definitions that validate at runtime, so an artifact that parses is an artifact the engine can run.

5.2 A step, annotated

```
{
  "id": "s3",
  "intent": "Enter the member number into the search field", // model's words, kept
  "action": "fill",
  "target": {
    "strategies": [ // ranked, tried in order
      { "kind": "role", "role": "textbox", "name": "Member Number" },
      { "kind": "labelText", "value": "Member Number" },
      { "kind": "nearText", "value": "Member Number", "direction": "right" },
      { "kind": "css", "value": "#ctl00_pnlSrch_txtMbr" }, // observed, brittle,
      { "kind": "bbox", "value": [412, 236, 180, 24] } // per-tenant patchable
    ],
    "elementDescription": "Member number input in the search panel",
    "framePath": [] // frameset-safe addressing
  },
  "value": "{{inputs.memberId}}", // literal 12345 parameterized away
  "waitBefore": { "condition": { "elementVisible": { "kind": "labelText",
    "value": "Member Number" } }, "timeoutMs": 5000 },
  "checkpoint": { "all": [ { "valueMatches": { "target": "self",
    "pattern": "{{inputs.memberId}}" } } ] },
  "risk": "safe"
}
```

5.3 Target descriptors: ranked, semantic-first locator strategies

Each target carries an ordered list of independent strategies, tried at replay time until exactly one node matches. The ranking encodes a robustness argument, not a preference:

Rank	Strategy	Why this position
1	role + accessible name	Most stable identity across styling, branding, and markup refactors; portable to desktop accessibility APIs; it is how a human names the control ("the Member Number textbox").
2	labelText (associated label)	Explicit label-control association; very stable on server-rendered form-heavy enterprise apps.
3	nearText (anchor text + direction)	Legacy tables often have no label association at all - the visible text next to the control is the only semantic anchor. Survives non-semantic markup; this is the no-clean-DOM workhorse.
4	relativeTo (structural relation)	For grids: "the balance cell in the row containing {{inputs.memberId}}". Anchors on data, not on markup positions.
5	css / xpath (as observed)	Precise and fast when it works, but brittle and tenant-specific (generated IDs). Recorded because it was true at discovery; the natural slot for per-tenant overrides (Section 10).
6	bbox + elementDescription	Universal last resort; also the bridge to screenshot/vision drivers and desktop surfaces. Description is for humans and vision models alike.

Uniqueness rule: ambiguity fails closed

A strategy that matches zero nodes or more than one node is treated as a miss and the next rank is tried; if all ranks miss, the step raises a deviation - the engine never guesses among candidates. In a banking back-office, acting on the wrong control is strictly worse than stopping. Replay telemetry records which rank matched (fallback depth), which becomes the drift signal in Section 10.3.

5.4 Conditions: a small declarative AST

Waits, checkpoints, outcome detectors, and recovery detectors all share one predicate language:

```
Condition :=
  { "all":    Condition[] }           // conjunction
  | { "any":  Condition[] }           // disjunction
  | { "not":  Condition  }
  | { "urlMatches": "<pattern>" }     // template-aware ("/members/{{inputs.memberId}}")
  | { "titleMatches": "<pattern>" }
  | { "elementVisible": TargetDescriptor | Strategy }
  | { "elementAbsent":  TargetDescriptor | Strategy }
  | { "textPresent":    "<string|pattern>" }           // page or scoped to a target
  | { "textAbsent":     "<string|pattern>" }
  | { "valueMatches":   { target, pattern } }         // input value assertions
```

- **Serializable and reviewable:** a checkpoint is data a reviewer can read, not code that must be trusted; no eval, no injection surface.
- **Surface-agnostic:** every predicate is expressible against web, desktop accessibility trees, or OCR/vision output - conditions do not name DOM concepts.
- **Composable:** outcome detectors are just conditions, so "no records found banner visible AND results table absent" is one declaration.

5.5 Parameterization and secrets

The Recorder rewrites every occurrence of a declared input's literal value (in fill values, URLs, and checkpoint patterns) into {{inputs.name}} references - which is simultaneously the reuse mechanism and the reason raw customer data never lands in an artifact. Credentials never appear even at discovery time: login steps reference {{secrets.tellerPassword}}, resolved from the environment at runtime, and marked so evidence masks them. Sensitive-flagged inputs and outputs are masked in every log line and screenshot (Section 8.3).

5.6 Versioning and review lifecycle

- **schemaVersion** versions the format (migrations); **capability.version** (semver) versions the flow - a locator patch is a patch bump, a changed contract (inputs/outputs/outcomes) is a major bump because callers depend on it.

- **appFingerprint** pins what the flow was recorded against; replay verifies it up front and fails fast with DRIFT_SUSPECTED rather than acting on a changed app.
- **reviewStatus: draft -> approved** is set by a human after reading the artifact (a git PR review of the JSON). Risky capabilities cannot replay unattended while in draft - the approval state is enforced by the policy engine, not by convention.

5.7 Alternatives rejected

Alternative	Why rejected
Record a runnable script (generated Playwright code)	Not reviewable as data, not parameterizable without parsing code, no seam for policy checks or outcome declarations, and locked to one surface technology. Code generation from the artifact is the right direction (a stretch goal), not the storage format.
Store the raw LLM transcript as the artifact	The brief explicitly requires decoupling. Transcripts are nondeterministic, unreviewable, and full of raw page content (PII). Provenance keeps a pointer to the run, not the transcript.
Single flat step list without declared outcomes/recoveries	Forces every deviation to be a failure - precisely the business-outcome/failure conflation the brief warns is the most common design mistake.
One monolithic locator per element	A single selector is a single point of failure and gives no cross-tenant story; ranked strategies degrade gracefully and make drift measurable.

6. Deterministic replay and the error taxonomy

Serves evaluation criteria: Correctness of the core loop, robustness and error handling (criteria 2-3).

Replay is the production path: given an artifact and typed params, execute the flow with **no model in the decision loop** and return a structured result. The engine's design goal is honesty under deviation - the brief is explicit that the interesting failures are runtime conditions, not layout drift, and that a replay must respond deliberately rather than blindly proceed.

6.1 Execution algorithm

```
replay(artifact, params):
    validate params against artifact.inputs (Zod)           // typed at the boundary
    policy.load(artifact.policy + global policy.yaml)
    driver.open(render(artifact.target.entrypoint))
    assert appFingerprint                                  // else DRIFT_SUSPECTED hard fail

    for step in artifact.steps:
        waitFor(step.waitBefore, bounded)                  // named condition, never sleep()
        guardScan()                                         // cheap global detectors: known
                                                             // dialogs, session-expiry marker
        handle = resolve(step.target)                      // ranked; unique-or-next-rank;
                                                             // records fallbackDepth

        driver.act(step.action, handle, render(step.value)) // PolicyEngine check inside act()
        ok = verify(step.checkpoint, bounded)
        if deviation (resolve miss | act error | checkpoint fail | guard hit):
            classify(snapshot):
                1. matches artifact.outcomes[].detector    -> return BUSINESS_OUTCOME(code)
                2. matches recoveries[].detector            -> apply recovery, retry step
                                                             (bounded by maxAttempts)
                3. else                                     -> evidence bundle;
                                                             escalate if policy allows;
                                                             else return HARD_FAILURE(step,
                                                             expected, observed, evidence)

    outputs = extract(artifact.outputs)                    // from declared source steps
    verify(artifact.successCheckpoint, bounded)            // else classify as above
    return SUCCESS(outputs, telemetry)
```

6.2 Wait and checkpoint strategy

- **No sleeps, ever.** Every wait is a named condition with a bounded timeout, recorded in the artifact from what discovery actually observed (URL changed, element appeared). Playwright's actionability checks (visible/enabled/stable) cover the last mile before each action.
- **A checkpoint after every step**, not only at the end. Rationale: it bounds the blast radius of a silent failure to one step, makes the failing step the unit of debugging (what was expected vs observed, right there), and gives the resume logic after human handoff a precise re-entry test (Section 7.5).
- **A final successCheckpoint** asserts the business goal itself (e.g. the balance panel for that member is visible) so success is verified, never assumed - the glossary's definition of a checkpoint.
- **Guard scans between steps** run a handful of cheap global detectors (known interstitial, session-expiry marker, error banner) so an unexpected dialog is caught at the seam where it appears, not three steps later as a confusing resolve failure.

6.3 The three-class taxonomy, applied to the brief's own conditions

Runtime condition	Class	Replay response	What the caller sees
Record / member not found	Business outcome	Detector matches; extract any declared fields; stop cleanly.	status=business_outcome, code=MEMBER_NOT_FOUND
Validation error on a form	Business outcome (declared)	Detector matches field-error state; extract the messages.	code=VALIDATION_REJECTED + messages

Runtime condition	Class	Replay response	What the caller sees
Permission denied	Business outcome (terminal)	Legitimate state the caller must know; never retried.	code=PERMISSION_DENIED
Known interstitial dialog (e.g. maintenance notice)	Recoverable	Declared recovery dismisses it; step retried; recovery logged.	Success (telemetry shows the recovery)
Transient slowness / failed load	Recoverable	Bounded wait-and-retry with backoff (maxAttempts).	Success, or hard failure once bounds exhaust
Session timeout / expiry	Recoverable (bounded) or escalate	Re-auth recovery if policy grants a secretRef; otherwise intervention.	Success after re-auth, or status=escalated
Unexpected unknown dialog	Hard failure -> escalate	No detector matches: never dismiss what we cannot name.	status=escalated (or hard_failure with evidence)
App error / HTTP 500	Hard failure	Evidence bundle (screenshot, a11y snapshot, URL, console); stop.	status=hard_failure with step, expected, observed
Ambiguous or missing target	Hard failure	All strategy ranks missed or matched multiple nodes; fail closed.	status=hard_failure (+ DRIFT_SUSPECTED hint)

Why classification runs in this exact precedence

Outcomes before recoveries: an expected business state must never be "recovered" away - retrying a not-found lookup is at best wasteful and at worst masks the caller's answer.

Recoveries before failure: only conditions someone declared and bounded (maxAttempts) are recoverable; recovery is a whitelist, not a heuristic.

Failure is the default: anything unrecognized fails closed with evidence. The classifier evaluates detectors against one stable snapshot so the decision itself is deterministic.

6.4 Result contract

```

type ReplayResult =
  | { status: "success";          outputs: Record<string, Value>;
    telemetry: RunTelemetry; evidenceRef: string }
  | { status: "business_outcome"; code: string;          // e.g. "MEMBER_NOT_FOUND"
    message: string; extracted?: Record<string, Value>; evidenceRef: string }
  | { status: "hard_failure";     error: { stepId: string; intent: string;
    expected: Condition; observed: StateSummary; evidencePaths: string[] };
    telemetry: RunTelemetry }
  | { status: "escalated";        intervention: { id: string; reason: string;
    resolution: "human_completed" | "human_fixed_resumed" | "aborted" };
    finalStatus: "success" | "business_outcome" | "aborted"; evidenceRef: string };

// RunTelemetry: per-step { strategyRankUsed, attempts, durationMs } + totals.
// The wrapper never throws for business conditions: outcomes are values, not errors.

```

6.5 What determinism means here

Same artifact + same params + same app state produces the same action sequence and the same classification. Controlled: no model calls (enforced by the import-boundary lint on replay/); branching only through declared detectors; fixed viewport, locale, and timezone on the browser context; template rendering is pure. Allowed to vary: timing within bounded waits and which locator rank matched - both are recorded as telemetry, not silent. UI drift - secondary per the brief - surfaces as rising fallback depth or a fingerprint mismatch and is reported as DRIFT_SUSPECTED rather than misclassified as a random failure.

7. Escalation and human-in-the-loop handoff

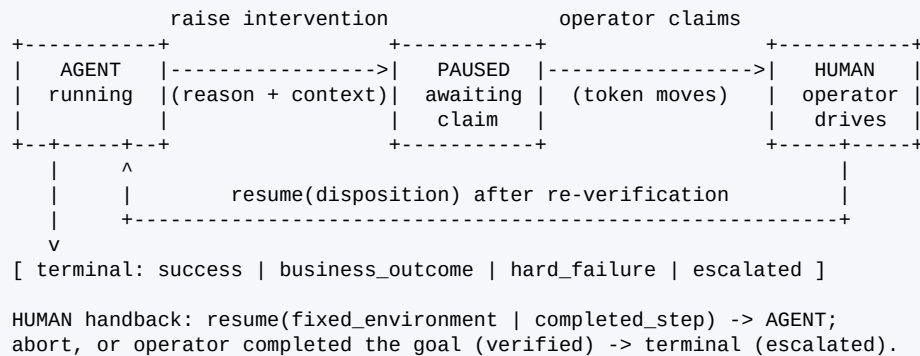
Serves evaluation criteria: Human-in-the-loop escalation (criterion 4) - a real mechanism, not a TODO.

The design centers on one idea: **control of a live session is an owned token with an auditable state machine**, not an ad-hoc pause. At any moment exactly one party may act - the automation or a named human - and every transition is logged with who, when, and why. The same mechanism serves both uses the brief names: stuck states and risky-action approvals.

7.1 Detecting "stuck"

Phase	Trigger
Discovery	Loop detection (same action + same state hash three times); max steps or wall-clock budget exceeded; the model itself calls escalate(reason); PolicyEngine blocks a risky action the model requested.
Replay	DeviationClassifier reaches class 3 (no outcome, recoveries exhausted or none apply); unknown dialog, failed checkpoint, ambiguous target, app error. Escalation is attempted before returning a hard failure when policy allows an operator.
Approval	A step marked risk=risky requires confirmation (discovery: always; replay: whenever unattendedReplay is false or reviewStatus is draft). Raised as an intervention of type approval - same pipeline, different disposition set (approve_once / deny).

7.2 Control-ownership state machine



The automation loop physically cannot act while it does not hold the token: the RunController exposes an awaitControl() gate the executor passes through before every step, and the driver tags every action with the current holder. This is the seam the brief asks about - pause, cede, resume on the same session - expressed as a mutex plus a state machine rather than convention.

7.3 The intervention request

```
{
  "id": "int_2026_0911_0142",
  "runId": "replay_a8c3",
  "capability": "member-savings-lookup@1.0.0",
  "phase": "replay",
  "stepId": "s2", "stepIntent": "Log in as the teller",
  "reason": "recovery_exhausted: session-expiry recovery failed twice",
  "classification": "hard_failure_candidate",
  "state": { "url": "http://localhost:4600/login?expired=1",
    "screenshot": "evidence/replay_a8c3/screens/012_intervention.png",
    "allSnapshots": "evidence/replay_a8c3/snapshots/012.json" },
  "requested": "fixed_environment", // what the system thinks it needs
  "controlToken": { "holder": "system", "since": "2026-09-11T01:42:07Z" },
  "expiresAt": "2026-09-11T02:12:07Z" // stale interventions abort safely
}
```

7.4 Taking control of the live session

- The browser runs **headed**; the session (cookies, storage, page state) is the one the automation was using - not a fresh context.
- The **operator console** (a deliberately minimal local web page, localhost:4700) lists open interventions with full context: capability, step and intent, reason, screenshot, current URL. Claiming one flips the state machine to HUMAN and hands the operator the token.
- The operator acts **directly in the same browser window**. An injected recorder (a page binding capturing clicks, fills, and navigations, with values redacted) writes each human action into the same evidence stream with actor="human" - so the record of what the human did survives the handoff, as required.
- **Production design (documented, not built)**: for remote operators the same token model rides on CDP screencast or noVNC to a browser pool; the console gains a live view. Nothing in the artifact, engine, or state machine changes - only the transport for human input. This is the deliberate mock seam the brief invites.

7.5 Handback and resume semantics

Disposition	Meaning	Engine behavior on resume
fixed_environment	"I repaired the state (e.g. re-logged-in); the step never completed."	Re-verify app fingerprint; re-run the current step from its waitBefore; continue normally.
completed_step	"I performed this step (perhaps several) manually."	Verify the current step's checkpoint; if it passes, advance; probe subsequent checkpoints to find the furthest verified step before continuing - never re-execute a mutating step a human already did.
approve_once / deny	Decision on a risky-action approval request.	Execute the gated step exactly once, or classify the run as blocked and stop with a clear result.
abort	"This run should not continue."	Terminal: result status escalated with resolution=aborted, evidence sealed.

Why checkpoints make resume safe

Resume never trusts the human's claim: it trusts the artifact's per-step checkpoints, evaluated against the live session. The human says what they intended; the engine verifies what is true. This is also why per-step checkpoints (Section 6.2) are non-negotiable in the schema: they are the re-entry tests for handoff, not just failure locators.

8. Safety and data handling

Serves evaluation criteria: Safety and data handling (criterion 6); also constrains discovery and replay design.

8.1 One chokepoint, physically unavoidable

The PolicyEngine is invoked **inside SurfaceDriver.act()** - not by the callers. Discovery, replay, and any future assisted-recovery path all act through the driver, so no code path can reach the UI without passing policy. This placement is the design's single most important safety property: enforcement by construction rather than by caller discipline. (The alternative - each runner checking policy before calling the driver - fails open the day someone adds a third caller.)

8.2 The allowlist model (deny by default)

- **Origins and routes:** navigation and actions are permitted only on origins/URL patterns listed in policy.yaml plus the artifact's requiredOrigins. Anything else is blocked and logged - during discovery the model is told its action was policy-blocked (it must find another way or escalate), during replay the block is a hard failure with evidence.
- **Action kinds:** the global config and each artifact declare which of navigate/click/fill/select/press/extract are permitted; e.g. a read-only capability that suddenly needs a submit fails policy, which catches both drift and tampering.
- **Budgets:** max steps, per-step and per-run timeouts, and max recovery attempts are policy, so runaway loops are structurally impossible.

8.3 Risky and irreversible actions

Every step carries risk: safe or risky, assigned at recording (mutating submits, anything past a confirmation screen) and reviewable. Handling is conservative and justified by asymmetry: a blocked action costs seconds of operator time; a wrong irreversible write in a banking system has unbounded cost. Concretely: during discovery, risky actions always require a human approval (the intervention pipeline); during replay, risky steps run unattended only when the artifact is reviewStatus=approved AND its policy sets unattendedReplay=true - otherwise the run pauses for approval. Reads never require approval.

8.4 Redaction pipeline (regulated financial data)

Layer	Mechanism
Artifacts	Recorder parameterization replaces literal input values with {{inputs.*}} references; sensitive example values are never stored; credentials exist only as {{secrets.*}} references resolved from the environment at runtime.
Logs	The evidence logger applies redaction before write: values of sensitive-flagged params/outputs and any {{secrets.*}} resolution are masked (never logged in the clear, anywhere).
Screenshots	Steps that touch sensitive-flagged fields are captured with Playwright's mask option over those locators; a per-step screenshot=off escape hatch exists for fully sensitive screens.
LLM transcript	Not persisted. Provenance stores model, provider, runId, and distilled per-step intents - the decoupling the brief requires - so page content containing PII does not outlive the run.
Repo hygiene	No secrets in the repo: .env.example documents required keys; the mock app ships only synthetic data.

8.5 Prompt-injection stance and honest limits

- Page content is **untrusted input**: the agent's system prompt instructs the model to treat on-screen text as data, but the real guarantee is structural - whatever the model asks for, the allowlist and risk gates hold at the chokepoint. A page cannot talk the system into acting off-policy; at worst it wastes bounded steps.
- **Limits stated plainly**: redaction is flag-driven, not a DLP scanner - an unflagged sensitive field would leak into evidence until review catches it; screenshot masking is best-effort on legacy layouts; the mock app's auth is synthetic. These are documented in REPORT.md's Safety section rather than papered over.

8.6 policy.yaml (excerpt; full file in Appendix C)

```
allowlist:
  origins: ["http://localhost:4600"]
  actionKinds: [navigate, click, fill, select, press, extract]
risk:
  riskyActionKinds: [submit]
  riskyUrlPatterns: ["**/subaccounts/new/confirm"]
  unattendedRiskyReplay: false      # artifact may not override to true
budgets: { maxSteps: 25, stepTimeoutMs: 15000, runTimeoutMs: 300000 }
redaction: { maskSecrets: always, maskSensitiveParams: always }
```

9. Evidence and observability

Serves evaluation criteria: Evidence requirement 3.5; also what makes every other claim verifiable.

Every run - discovery, replay, or escalated - writes one self-contained directory: a structured JSONL event log (what happened and, during discovery, why the model chose it), screenshots at each step, checkpoint, and failure, machine-readable results, and any intervention records. The log is append-only and every entry is attributed to an actor, so a run that crossed a human handoff reads as one continuous, auditable story. All writes pass through the redacting logger (Section 8.4).

```
evidence/
disc_7f2e/          # discovery run (committed as required proof)
  run.jsonl          # every event, see record shape below
  screens/001_observe.png ... # per step + checkpoints + failure
  artifact.draft.json # what the Recorder emitted
  result.json
replay_a8c3/        # happy-path replay
replay_b911/        # MEMBER_NOT_FOUND business outcome
replay_c04d/        # injected session expiry -> escalated -> resumed
  interventions/int_...json # request + claim + dispositions
  run.jsonl          # includes actor:"human" entries from the recorder

run.jsonl record:
{ ts, runId, actor: "agent|"replay|"human|"policy",
  phase, stepId?, event, action?, target?, outcome?,
  reasoning?,           // model intent during discovery (redacted)
  durationMs?, evidence? // relative paths to screenshots/snapshots
}
```

The four directories named above are exactly what gets committed to /evidence/ for the graders: the real discovery run the brief demands, plus the three replay scenarios that exercise the taxonomy (success, business outcome, escalation with human resume). A Playwright trace.zip per run is kept locally and referenced, committed only if size-reasonable.

10. Heterogeneity and multi-tenant reuse (design)

Serves evaluation criteria: Generalization to the real environment (criterion 5). Design-only by the brief's instruction; the seams are built, the extra drivers are not.

10.1 The surface seam

One sentence carries the whole abstraction: **the artifact records what to find and verify (semantic descriptors and declarative conditions); the driver owns how to perceive and act.** Nothing in the artifact names a DOM, a selector engine, or a browser - so extending to a new surface means writing a driver, not re-recording capabilities.

```
interface SurfaceDriver {
  open(entrypoint: string): Promise<void>;
  observe(): Promise<UiSnapshot>;           // { url?, title?, nodes: UiNode[] }
  resolve(t: TargetDescriptor): Promise<Resolved | Ambiguous | NotFound>;
  act(a: StepAction, h?: NodeHandle): Promise<ActionResult>; // policy checked inside
  screenshot(mask?: TargetDescriptor[]): Promise<Buffer>;
  capabilities(): { hasA11yTree: boolean; hasDomFallback: boolean;
    coordinateActions: boolean };
}

interface UiNode {           // the surface-agnostic observation unit
  ref: string; role: string; name?: string; value?: string;
  states: string[]; bbox: [x: number, y: number, w: number, h: number];
  framePath: string[];       // window/frame/pane addressing
}
```

Surface	How the same artifact executes
Modern web (built)	PlaywrightDriver: a11y snapshot for observe; role/label/text engines for resolve; DOM events for act.

Surface	How the same artifact executes
Legacy web (designed; the mock app approximates it)	Same driver - resolution simply lands lower in the strategy ranking (nearText, relativeTo, css, bbox) because roles/labels are sparse. framePath addresses framesets. No schema change.
Desktop (designed)	UIA (Windows) / AXUIElement (macOS) driver: the OS accessibility tree maps onto the same UiNode shape - the glossary itself notes a11y trees exist on desktop. role+name strategies carry over; css ranks are skipped via capabilities().
Screenshot-only / hostile (designed)	Vision driver: observe via OCR + detection into UiNode[]; resolve uses elementDescription + bboxHint (recorded for exactly this); act by coordinates. A model assists perception only - decisions stay deterministic, and the policy chokepoint is unchanged.

10.2 Multi-tenant reuse: base artifact + tenant binding overlay

Hundreds of institutions run the same vendor product, differently branded and versioned. Re-recording per tenant is the failure mode to avoid, so the unit of recording is the **vendor product** (base artifact), and each tenant contributes only a thin, reviewable **binding**:

```
{
  "tenantId": "cu-eastshore",
  "appId": "vendorcore-teller", "appVersion": "9.4.x",
  "binding": {
    "entrypoint": "https://teller.eastshore.example/login",
    "secretsRef": "vault://eastshore/teller-bot",
    "vocabulary": { "Member": "Customer" },          // applied to name/label/text strategies
    "locatorOverrides": [
      { "stepId": "s3", "prepend": { "kind": "css", "value": "#custNum" } }
    ],
    "flags": { "hasMaintenanceInterstitial": true }
  }
}
```

- **Merge at load:** replay resolves base + binding into an effective artifact; overrides are per-step patches (prepend/replace a strategy), the vocabulary map rewrites name/label/text strategy values, and the base is never mutated - one upgrade path, many tenants.
- **Why semantic-first locators make this work:** role/label/nearText survive branding and theming; only the brittle css rank routinely needs a tenant patch - which is precisely where the override slot sits.
- **Safe specialization:** bindings carry no flow logic - a tenant cannot accidentally fork behavior, only identification and configuration. A tenant needing a different flow is a new capability version, reviewed like any other.

10.3 Detecting and managing drift

- **Telemetry as sensor:** every replay records, per step, which strategy rank matched (fallbackDepth), attempts, and timings. Aggregated per tenant/app-version, rising fallback depth or a new failure signature is the early-warning "locator health" signal - drift is observed from production replays, not discovered by outages.
- **Fail fast on version drift:** appFingerprint mismatch at preflight stops the run with DRIFT_SUSPECTED before any action - never act on an app that is not what was recorded.
- **Repair loop (design):** degraded capabilities get a canary re-validation; a bounded, policy-checked LLM pass may propose a locator patch as a diff for human review (the brief's assisted-fallback stretch) - the model proposes, a person approves, replay stays deterministic.

11. The mock target application ("CU BackOffice")

Serves evaluation criteria: Enables the robustness, safety, and escalation demonstrations; stands in for the real environment per brief Section 4.

A small Express + EJS server-rendered app with deliberately hostile, legacy-style markup: nested table layouts, no test IDs, generated-looking IDs (ctl00_pnlSrch_txtMbr), sparse label associations, inline styles, and a frameset-style shell on the detail screen. Synthetic data only. It runs offline; reviewers never need external services (the LLM key is needed only for discovery).

11.1 Flows

- **Login** - teller credentials (synthetic), session cookie with expiry.
- **Member search** - by member number; not-found is a natural state.
- **Member detail** - balances table (savings, checking) inside nested tables/frames; permission-restricted member 40001 renders an access-denied screen.
- **Open sub-account** - multi-field form, validation errors, review screen, then a mutating confirm that issues a confirmation number - the risky-action demo.

11.2 Fault-injection matrix (query flag or config)

Injection	App behavior	Exercises	Expected system response
(none) bad member id	Empty result + "No records found"	Business outcome	MEMBER_NOT_FOUND returned as a value
validation	Form rejects with field messages	Business outcome	VALIDATION_REJECTED + extracted messages
denied (member 40001)	Access-denied screen	Business outcome	PERMISSION_DENIED, terminal
interstitial	Maintenance modal before content	Recoverable	Declared recovery dismisses; step retried; logged
slow	3-8s delayed responses	Recoverable	Bounded wait/retry within budgets
expire	Session cookie invalidated mid-flow	Recoverable -> escalate	Re-auth recovery; if disabled by policy, intervention -> human login -> resume
error500	Server error page	Hard failure	Evidence bundle; structured hard_failure

Demo capabilities recorded against it: **member-savings-lookup** (read-only; the committed discovery run) and **open-sub-account** (contains a risky step; demonstrates the approval gate). A tenant-B skin (different branding, "Customer" vocabulary, one changed control id) is the cross-tenant stretch demo.

12. Testing strategy - where it counts

Serves evaluation criteria: Code quality (criterion 7): "tested where it counts", not blanket coverage.

Layer	What is tested and why it is the right thing
Unit: artifact schema	Zod round-trips (parse-serialize-parse), rejection of malformed artifacts, template rendering incl. escaping - the contract everything else trusts.
Unit: policy engine	Allow/deny matrix over origins, action kinds, risk gates, approval states - the safety claims, verified.
Unit: deviation classifier	Precedence (outcome beats recovery beats failure), bounded retries, unknown-state fail-closed - the taxonomy the rubric weighs.
Unit: locator resolution	Fixture HTML pages: rank order, uniqueness rule, fallback-depth telemetry, frame addressing.
Unit: redaction	Sensitive params/secrets never appear in log output or serialized artifacts (asserted by scanning).
Integration (vitest + Playwright vs the mock app)	One test per taxonomy row of Section 11.2, plus the happy path and an escalate-and-resume walkthrough with a scripted "operator" driving the console API.

Deliberately untested: EJS templates, CLI arg parsing, console styling - low-risk glue. The LLM call itself is tested with a recorded-response fake; the one real discovery run is the live proof, committed as evidence.

13. Repository layout and deliverables mapping

Serves evaluation criteria: Communication (criterion 8) and the brief's exact deliverable paths.

```
/                # public GitHub repo root
README.md        # setup, keys, demo path (exact commands)
REPORT.md        # the 7 required headings, 1-3 pages
policy.yaml      # global allowlist and budgets
.env.example     # ANTHROPIC_API_KEY / OPENAI_API_KEY (discovery only)
artifacts/       # saved capabilities (reviewable JSON, versioned names)
evidence/        # committed runs: disc_*, replay_* (Section 9)
src/
  core/ surface/ llm/ agent/ replay/
  policy/ escalation/ evidence/ cli/
src/target-app/  # mock CU back-office (standalone)
test/           # unit + integration
```

REPORT.md uses the seven mandated headings verbatim - Architecture, Artifact schema, Determinism and error handling, Heterogeneity and multi-tenant, Escalation and handoff, Safety, Cuts - each a distilled page-share of this document's Sections 3, 5, 6, 10, 7, 8, and 14 respectively. Replay runs with no API key; only discovery needs one, and README says so up front.

13.1 Demo path (verbatim in README)

```
npm install && npx playwright install chromium
npm run target                # mock app on http://localhost:4600
npm run operator              # operator console on http://localhost:4700

npm run discover -- --goal "Look up member 12345 and read their current savings \
balance" --entry http://localhost:4600 --param memberId=12345 \
--probe memberId=99999       # discovery + outcome probe -> artifacts/...

npm run replay -- -c member-savings-lookup -p memberId=12345          # success
npm run replay -- -c member-savings-lookup -p memberId=99999          # business outcome
npm run replay -- -c member-savings-lookup -p memberId=12345 --inject interstitial
npm run replay -- -c member-savings-lookup -p memberId=12345 --inject expire
                                # -> intervention -> take over in the headed
                                # browser -> resume from the console
```

14. Implementation roadmap, cut lines, and stretch goals

Serves evaluation criteria: Everything - this is the order in which the criteria get built and verified.

14.1 Phases (est. 28-36 focused hours total)

Ph	Build	Exit criterion (definition of done)	Hrs
0	Scaffold: strict TS, ESLint + dependency-boundary rule, vitest, prettier	CI-green empty skeleton; replay/ -> llm/ import fails lint	1-2
1	core/: artifact schema, condition AST, result contract, policy config (Zod)	Appendix A artifact parses; malformed fixtures rejected; types inferred	3-4
2	target-app/: flows, legacy markup, fault injection, seed data	All Section 11.2 states reachable by hand in a browser	3-4
3	surface/: PlaywrightDriver - a11y observe, ranked resolve, act with policy chokepoint, screenshots+masking	Driver test-page suite green incl. frames and ambiguity fail-closed	4-5
4	policy/ + evidence/: engine, redacting JSONL logger, run dirs	Policy matrix tests green; secret-scan test proves no leaks	2-3
5	llm/ + agent/: both providers, discovery loop, stop conditions, Recorder with parameterization + probe runs	Real discovery completes the lookup goal; artifact emitted and valid	5-6
6	replay/: executor, waits, checkpoints, classifier, recoveries, result contract	All taxonomy integration tests green; no-API-key replay verified	5-6
7	escalation/: RunController, intervention store, operator console, human-action recorder, resume	expire scenario: pause -> human login in same browser -> resume -> success, fully evidenced	4-5
8	cli/ + evidence capture: the four committed runs; README demo path verified from a clean clone	A reviewer can reproduce replay demos with three commands	2-3
9	REPORT.md (7 headings), README polish, final read-through	Docs match behavior; cut lines explicit	2-3

14.2 Cut lines - deliberate, with the seam designed

- **Operator console is minimal by intent** (list, claim, resume, dispositions): the brief scopes out a real co-browsing console; the control-transfer model is the graded part and it is fully real.
- **Remote session streaming (CDP screencast / noVNC)**: designed in Section 7.4, not built - local headed browser makes the handoff real without transport work.
- **Desktop and vision drivers**: interface + design only (Section 10.1); the brief says design, not build.
- **Multi-tenant bindings**: schema + merge design (Section 10.2); built only if the stretch demo is attempted.
- **Assisted LLM recovery on replay failure**: designed as a propose-a-patch flow (10.3), not built - keeps the replay path pure.

14.3 Stretch goals - at most two, in this order

- **1. Agent-facing capability catalog**: a catalog command / tiny endpoint listing artifacts as callable tools (name, description, JSON-schema params derived from inputs) plus one scripted invocation - it closes the brief's through-line loop (the artifact as an agent-invokable capability) for a few hours of work.
- **2. Tenant-B variant**: re-skinned mock app + a TenantBinding making the same base artifact pass on both - turns the Section 10 story into a demonstration.

14.4 What gets built next with more time

Confidence scoring from replay telemetry gating unattended runs; the assisted-recovery patch proposer; artifact registry with signed approvals; remote operator transport; a desktop (UIA) driver against a sample WinForms app; canonicalization of routes into parameterized patterns across capabilities.

15. Risk register

Risk	L	I	Mitigation
Discovery flakiness (model meanders, mislabels targets)	M	M	Closed tool set, temperature 0, compact snapshots, loop detector, probe-run design; only a verified successful run is distilled; retry runs are cheap by design.
Hostile markup yields poor accessible names	M	M	nearText/relativeTo/bbox ranks exist for exactly this; the mock app keeps visible text anchors (as real back-office apps do).
Human-action recording rabbit hole	M	L	Scope pinned to clicks/fills/navigations via one injected binding; anything deeper is out of scope and documented.
Handoff race conditions (agent acts while human drives)	L	H	Single control token checked in awaitControl() before every step and asserted inside driver.act(); transitions are atomic in one process.
Sensitive data slips into evidence	L	H	Redaction at the logger boundary (single write path), secret-scan unit test, screenshot masking, parameterized artifacts.
Scope creep vs the time box	M	M	Phase exit criteria above; cut lines pre-declared; stretch goals gated on a finished core.

L = likelihood, I = impact (H/M/L).

Appendix A - Complete example artifact (member-savings-lookup)

Internally consistent with Sections 5, 6, and 11; this is the shape the Recorder emits and the fixture the schema tests parse.

```
{
  "schemaVersion": "1.0",
  "capability": {
    "id": "member-savings-lookup",
    "name": "Look up member savings balance",
    "description": "Finds a member by member number and returns the current savings balance shown on the member detail screen.",
    "version": "1.0.0",
    "tags": ["read-only", "member-servicing"]
  },
  "target": {
    "appId": "cu-backoffice",
    "surface": "web",
    "entrypoint": "{{env.APP_BASE_URL}}/login",
    "appFingerprint": {
      "titlePattern": "CU BackOffice - Teller",
      "markers": [{ "textPresent": "CU BackOffice v2" }]
    }
  },
  "inputs": [
    { "name": "memberId", "type": "string", "required": true, "pattern": "^\\d{5}$", "sensitive": false, "description": "Five-digit member number", "example": "12345" }
  ],
  "outputs": [
    { "name": "savingsBalance", "type": "money", "sensitive": true, "description": "Current savings balance", "sourceStep": "s7" },
    { "name": "memberName", "type": "string", "sensitive": true, "description": "Member display name", "sourceStep": "s6" }
  ],
  "steps": [
    { "id": "s1", "intent": "Open the teller login page", "action": "navigate", "value": "{{env.APP_BASE_URL}}/login", "checkpoint": { "all": [{ "elementVisible": { "kind": "role", "role": "button", "name": "Sign In" } } ] }, "risk": "safe" },
    { "id": "s2", "intent": "Log in as the teller", "action": "fill+submit", "target": { "strategies": [ { "kind": "labelText", "value": "Operator ID" }, { "kind": "css", "value": "#ctl00_login_txtOp" } ] }, "elementDescription": "Operator ID field on login form", "framePath": [] }, "value": { "operator": "{{secrets.tellerId}}", "password": "{{secrets.tellerPassword}}", "checkpoint": { "all": [{ "urlMatches": "/home", { "textPresent": "Member Services" } } ] }, "risk": "safe" },
    { "id": "s3", "intent": "Enter the member number into the search field", "action": "fill", "target": { "strategies": [ { "kind": "role", "role": "textbox", "name": "Member Number" }, { "kind": "labelText", "value": "Member Number" }, { "kind": "nearText", "value": "Member Number", "direction": "right" }, { "kind": "css", "value": "#ctl00_pnlSrch_txtMbr" },
```

```

    { "kind": "bbox", "value": [412, 236, 180, 24] }},
    "elementDescription": "Member number input in the search panel",
    "framePath": [ ] },
    "value": "{{inputs.memberId}}",
    "waitBefore": { "condition": { "elementVisible":
      { "kind": "labelText", "value": "Member Number" } }, "timeoutMs": 5000 },
    "checkpoint": { "all": [ { "valueMatches":
      { "target": "self", "pattern": "{{inputs.memberId}}" } } ] },
    "risk": "safe" },
    { "id": "s4", "intent": "Run the search", "action": "click",
    "target": { "strategies": [
      { "kind": "role", "role": "button", "name": "Search" },
      { "kind": "css", "value": "#ctl00_pnlSrch_btnGo" }],
      "elementDescription": "Search button", "framePath": [ ] },
    "checkpoint": { "any": [
      { "elementVisible": { "kind": "nearText", "value": "Member Results" } },
      { "textPresent": "No records found" } ] },
    "risk": "safe" },
    { "id": "s5", "intent": "Open the member detail record", "action": "click",
    "target": { "strategies": [
      { "kind": "relativeTo",
        "anchor": { "kind": "text", "value": "{{inputs.memberId}}" },
        "relation": "row-link" },
      { "kind": "css", "value": "table#ctl00_grdRes a" }],
      "elementDescription": "Result row link for the searched member",
      "framePath": [ ] },
    "checkpoint": { "all": [
      { "urlMatches": "/members/{{inputs.memberId}}" } ] },
    "risk": "safe" },
    { "id": "s6", "intent": "Read the member name", "action": "extract",
    "target": { "strategies": [
      { "kind": "nearText", "value": "Name:", "direction": "right" }],
      "elementDescription": "Member name value on detail header",
      "framePath": ["detailFrame"] },
    "extract": { "name": "memberName", "type": "string" },
    "checkpoint": { "all": [ { "textPresent": "Accounts" } ] },
    "risk": "safe" },
    { "id": "s7", "intent": "Read the savings balance from the accounts table",
    "action": "extract",
    "target": { "strategies": [
      { "kind": "relativeTo",
        "anchor": { "kind": "text", "value": "Savings" },
        "relation": "row-cell", "cellHint": "Balance" },
      { "kind": "bbox", "value": [388, 402, 110, 20] }],
      "elementDescription": "Balance cell of the Savings row",
      "framePath": ["detailFrame"] },
    "extract": { "name": "savingsBalance", "type": "money" },
    "checkpoint": { "all": [ { "valueMatches":
      { "target": "self", "pattern": "^\\$[0-9,]+\\.\\d{2}$" } } ] },
    "risk": "safe" }
  ],
  "outcomes": [

```

```

{ "code": "MEMBER_NOT_FOUND", "terminal": true,
  "description": "No member exists with the supplied member number.",
  "detector": { "all": [{ "textPresent": "No records found" },
    { "elementAbsent":
      { "kind": "nearText", "value": "Member Results" } } ] } },
{ "code": "PERMISSION_DENIED", "terminal": true,
  "description": "Teller lacks access to this member record.",
  "detector": { "textPresent": "You do not have permission" } }
],
"recoveries": [
  { "id": "maintenance-interstitial", "appliesTo": "global", "maxAttempts": 2,
    "detector": { "elementVisible":
      { "kind": "role", "role": "dialog", "name": "System Notice" } },
    "action": { "kind": "dismiss", "target": { "strategies": [
      { "kind": "role", "role": "button", "name": "OK" } ] } } },
  { "id": "transient-slow-load", "appliesTo": "global", "maxAttempts": 2,
    "detector": { "textPresent": "Loading..." },
    "action": { "kind": "waitRetry", "backoffMs": 2000 } },
  { "id": "session-expiry-reauth", "appliesTo": "global", "maxAttempts": 1,
    "detector": { "urlMatches": "/login\\?expired=1" },
    "action": { "kind": "runSteps", "stepIds": ["s2"] } }
],
"successCheckpoint": { "all": [
  { "urlMatches": "/members/{{inputs.memberId}}" },
  { "textPresent": "Accounts" } ] },
"policy": {
  "requiredOrigins": ["{{env.APP_BASE_URL}}"],
  "allowedActionKinds": ["navigate", "click", "fill", "extract"],
  "riskLevel": "readonly",
  "unattendedReplay": true
},
"provenance": {
  "provider": "anthropic", "model": "claude-sonnet-4-6",
  "runId": "disc_7f2e", "recordedAt": "2026-09-10T18:20:11Z",
  "evidenceRef": "evidence/disc_7f2e/", "reviewStatus": "approved"
}
}

```

Appendix B - Result contract examples

```
// success
{ "status": "success",
  "outputs": { "savingsBalance": "$4,912.55", "memberName": "****REDACTED****" },
  "telemetry": { "steps": 7, "retries": 0, "maxFallbackDepth": 1,
    "durationMs": 8412 },
  "evidenceRef": "evidence/replay_a8c3/" }

// business outcome (bad input 99999)
{ "status": "business_outcome", "code": "MEMBER_NOT_FOUND",
  "message": "No member exists with the supplied member number.",
  "evidenceRef": "evidence/replay_b911/" }

// hard failure (injected error500)
{ "status": "hard_failure",
  "error": { "stepId": "s4", "intent": "Run the search",
    "expected": { "any": [{ "elementVisible": "Member Results" },
      { "textPresent": "No records found" }] },
    "observed": { "url": "/members/search", "title": "Server Error",
      "topText": "HTTP 500 - internal error" },
    "evidencePaths": ["evidence/replay_x/screens/008_failure.png",
      "evidence/replay_x/snapshots/008.json"] },
  "telemetry": { "steps": 4, "retries": 2, "maxFallbackDepth": 1 } }

// escalated (injected session expiry, human logged in, run resumed)
{ "status": "escalated",
  "intervention": { "id": "int_2026_0911_0142",
    "reason": "recovery_exhausted: session-expiry recovery failed",
    "resolution": "human_fixed_resumed" },
  "finalStatus": "success",
  "evidenceRef": "evidence/replay_c04d/" }
```

Appendix C - policy.yaml

```
# policy.yaml - global guardrails (deny by default; artifacts may narrow,
# never widen, what this file allows)
allowlist:
  origins:
    - "http://localhost:4600"          # mock CU back-office
  routes:
    deny: ["**/admin/**"]              # never touched, even inside the origin
  actionKinds: [navigate, click, fill, select, press, extract, dismiss]

risk:
  riskyActionKinds: [submit]
  riskyUrlPatterns: ["**/subaccounts/new/confirm"]
  discoveryRiskyActions: require_approval # always a human decision
  unattendedRiskyReplay: false           # approved artifacts still need
                                          # policy to opt in per capability

budgets:
  maxSteps: 25
  stepTimeoutMs: 15000
  runTimeoutMs: 300000
  maxRecoveryAttemptsPerStep: 2

escalation:
  interventionTtlMinutes: 30            # stale interventions abort safely
  operatorConsole: "http://localhost:4700"

redaction:
  maskSecrets: always
  maskSensitiveParams: always
  screenshotMaskSensitiveFields: true
```